

基于 Spark 的深度学习文本分类方法在 GLUE 数据集上的应用研究

摘要：本文针对通用语言理解评估（GLUE）数据集，提出了一种结合分布式数据处理与预训练语言模型的文本分类方法。基于 Spark 框架对多任务文本数据进行并行预处理与分词转换，提高了大规模数据的处理效率。采用 DistilBERT 模型对 GLUE 各子任务进行迁移学习训练，针对任务差异设计了灵活的标签映射与评估策略。实验结果表明，该方法在多个文本分类任务上取得了良好的性能表现，验证了分布式数据处理与预训练模型结合的有效性与可扩展性。

关键词：Spark，分布式数据处理，文本分类，DistilBERT

1 引言

近年来，随着预训练语言模型（Pretrained Language Models, PLMs）在自然语言处理（Natural Language Processing, NLP）领域的广泛应用，文本分类任务的性能得到了显著提升。特别是 BERT、RoBERTa、DistilBERT 等模型的提出，使得以往依赖大量人工特征的文本处理方式逐步被深度学习驱动的端到端模型所取代。通过在大规模无标注语料上进行预训练，这类模型能够捕捉丰富的上下文语义信息，进而在各类下游任务中展现出强大的迁移能力。通用语言理解评估（General Language Understanding Evaluation, GLUE）基准集的提出，为衡量模型在多种语言理解任务上的泛化能力提供了统一的标准。GLUE 包含多个子任务，涵盖句子级分类、句对关系判断以及语义相似度计算等多种语言现象，已成为当前评估自然语言处理模型能力的主流基准。

本研究基于 DistilBERT 预训练语言模型，构建了一个统一的文本分类训练框架，针对 GLUE 数据集中的典型任务进行了微调实验。为提高训练效率和可扩展性，实验过程中引入了分布式计算框架 Spark，并在多个任务之间共享模型结构以实现参数迁移和特征共享。近几年，以 Spark 和 MapReduce 为代表的大数据分析技术的发展，为解决集成学习中单点算力的痛点带来了曙光^[1]。通过在不同子任务上的性能比较，进一步验证了预训练模型在跨任务迁移学习中的有效性与鲁棒性。本研究的主要目标在于探索轻量级预训练模型在多任务文本分类场景下的实际应用效果，并评估其在标准数据集上的性能表现，为构建通用语言理解系统提供实践依据和技术支持。

2 集成学习与 Spark 平台

2.1 文本情感

文本情感分析是自然语言处理中一个非常重要和实用方法。目前针对文本的情感倾向分析方法主要有两种：基于情感词典的方法和基于机器学习的方法。两种方法各有优劣，情感词典的方法主要优点是稳定，常常根据情感词就可以推断出整个文本的情感倾向，缺点是对于不在情感词典中的词无能为力，而且无法结合语义上下文的语境，而同一个词在不同语境中可能完全表达相反的含义^[2]。机器学习方法常常需要人工标注语料作为训练集，提取出文本的特征，用特征构建一个分类器，再做情感的分类。但是这种方法因为机器学习中单个模型特征的抽取方法不同，通用性不好，而且抽取过程中受训练语料噪音的影响进而导致分类效果并不太理想^[3]。

2.2 集成学习

集成学习是一种先进的机器学习策略，它通过整合多个独立训练的基学习器的预测结果，来构建一个更为强大、预测能力更优的强学习器。这种方法的核心优势在于能够有效降低单一模型的偏差和方差，从而显著提升整体模型的泛化能力、鲁棒性以及预测精度。集成学习方法主要分为两大类。首先是并行式集成 (Bagging)，这类方法鼓励基学习器独立训练。典型的例子包括 Bagging 本身和随机森林 (Random Forest)，它们通过对原始数据集进行有放回抽样 (bootstrap sampling) 生成多个不同的训练子集，每个子集训练一个基学习器，最后将所有基学习器的预测结果进行平均或投票以得到最终预测。其次是串行式集成 (Boosting)，与并行式不同，这类方法以迭代方式训练基学习器，每个新的基学习器都会侧重于纠正前一个学习器预测错误的样本，代表性算法有 AdaBoost、梯度提升决策树 (Gradient Boosting Decision Tree, GBDT) 和 XGBoost 等。

2.3 Spark 平台

Apache Spark 是一个专为大规模数据处理而设计的统一分析引擎。它提供了一个快速且通用的集群计算框架，旨在克服传统 Hadoop MapReduce 在迭代式计算和交互式查询方面的局限性。Spark 最显著的优势在于其强大的内存计算能力，这使得数据处理速度得以大幅提升。

Spark 支持多种主流编程语言的 API，包括 Scala、Java、Python 和 R，并配备了一系列功能丰富的库，能够胜任多样化的数据处理任务，例如：Spark SQL 专注于处理结构化数据；Spark Streaming 用于实时流数据处理；MLlib 专为机器学习任务设计；以及 GraphX 用于图计算。凭借其卓越的分布式特性，Spark 能够高效地在集群环境中并行处理海量数据。因此，它被广泛应用于大数据分析、

机器学习模型训练、ETL（Extract, Transform, Load）操作以及实时数据处理等关键领域。图 1 展示了 Spark 的基本框架。用户首先通过 `spark-submit` 命令向集群提交作业，随后 Driver 向 Master 注册该作业并申请资源。由于 Master 与各个 Worker 之间通过心跳包报告机器资源状况，Master 能够找到最适合的机器资源分配给当前任务。接着，在各个有资源的节点上启动 Executor 进程，这些进程负责运行用户提交任务中的每一个 Task。Executor 通过心跳包向 Driver 报告运行状态，直至作业完成。像 Spark 这类分布式计算框架的瓶颈通常在于网络 I/O。然而，在集成学习中，模型切分后单个模型之间相对独立，无需通过网络与其他节点进行数据交互，因此能够充分发挥集群机器的计算性能^[4]。

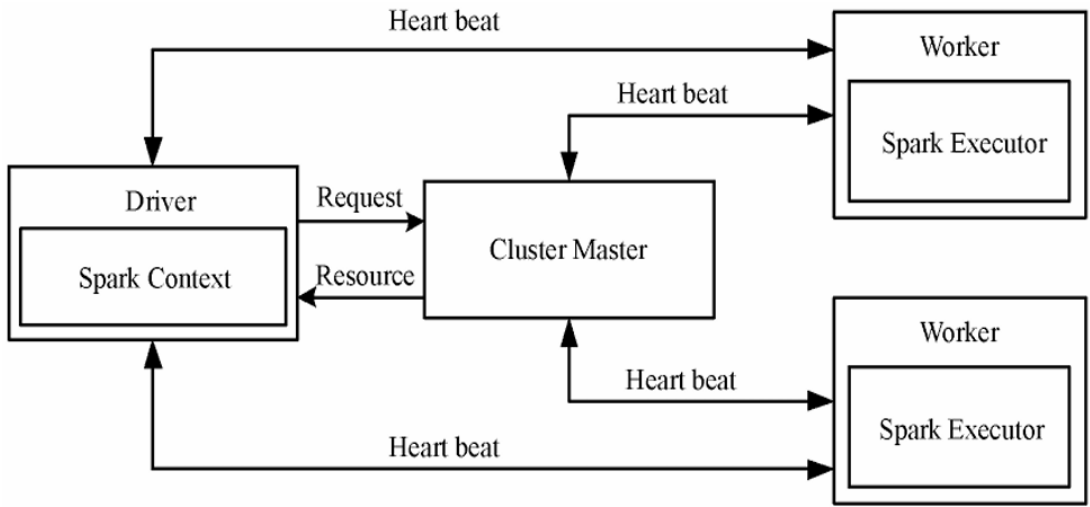


图 1 Spark 框架的工作流程

3 深度学习模型

DistilBERT 是由 Hugging Face 团队提出的一种轻量级预训练语言模型，基于 Transformer 编码器架构，采用知识蒸馏（knowledge distillation）技术将大型的 BERT 模型压缩而成。相比标准的 BERT-base，DistilBERT 仅有 6 层 Transformer（BERT-base 有 12 层），参数量减少到约 6600 万，约为 BERT-base 的 60%，这使得模型在保持较好语言理解能力的同时，显著提升了推理速度和降低了计算资源消耗。该模型采用无大小写区分的词汇表（uncased），更适合英文文本的通用处理。图 2 展示了 DistilBERT 模型的网络结构图，清晰地阐释了知识蒸馏训练学生模型的流程。在此过程中，学生模型的核心目标是学习并模仿教师模型的行为。它不仅仅通过交叉熵损失函数来学习真实标签，更为关键的是，它还通过 KL 散度损失来模仿教师模型的输出概率分布，即软目标。此外，图中所示的余弦相似度损失，作为一种可选的辅助机制，进一步促使学生模型学习教师模型在中间层产生的特征表示。这种将多重损失函数相结合的策略，形成了多

任务或多目标优化，使得学生模型在压缩模型体积的同时，依然能够保持卓越的性能。图中各损失项旁标注的数字，例如 5、1、2，极可能代表了这些不同损失项在最终总损失函数中的权重比例，从而调控着学生模型学习的不同侧重。

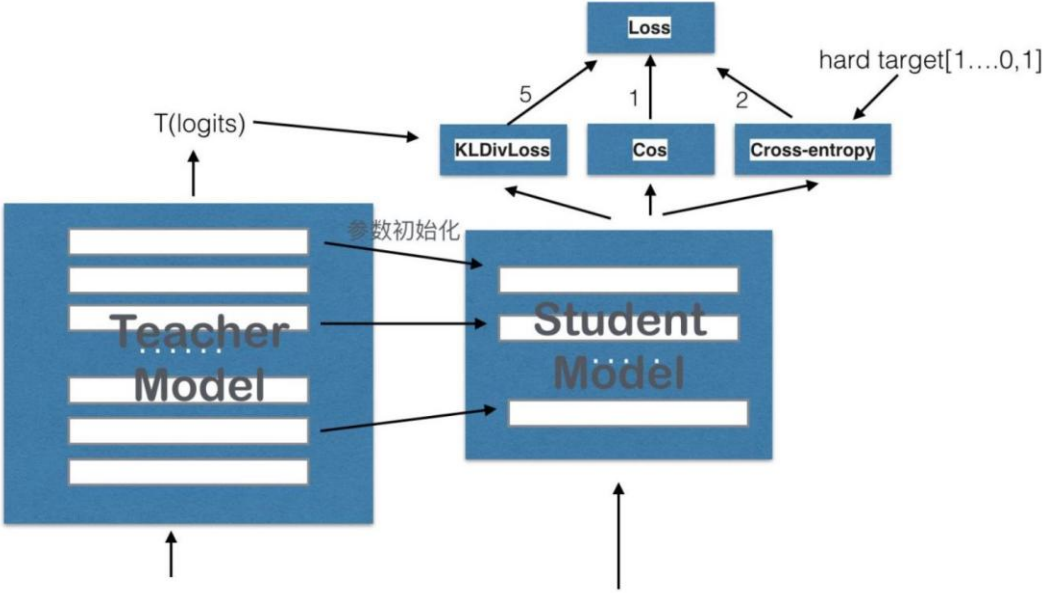


图 2 DistilBERT 模型的网络结构图

4 实验设计

4.1 数据集

通用语言理解评估 (GLUE) 基准测试是一项旨在评估自然语言处理 (NLP) 模型在多种语言理解任务上泛化能力的综合性标准。该基准测试由一系列精心挑选的、异构的英文自然语言理解子任务构成，这些任务涵盖了不同的数据规模、文本体裁和认知复杂度，从而为评估模型对语言细微差别的理解能力提供了全面的视角。GLUE 数据集的设立，旨在推动研究者开发出更具通用性和鲁棒性的自然语言理解系统，而非仅仅在单一任务上表现卓越的“专家”模型。其核心目标是鼓励模型学习和利用跨任务的共享语言知识，以提高在资源有限或领域迁移场景下的性能。

本实验的目的与 GLUE 基准测试的初衷高度契合。通过对 DistilBERT 模型进行微调，实验旨在探究该轻量级预训练模型在 GLUE 各项任务上的适应性和性能表现。GLUE 提供了多样化的任务类型，这使得 DistilBERT 模型能够在统一的框架下，接受多方面的语言理解能力考验。表 1 详细列举了 GLUE 基准测试中的各项任务，便于理解和选择合适的任务来评估自然语言处理模型。

表 1 GLUE 基准测试任务概览

名称	任务类型	目的	训练集	验证集
CoLA	单句分类	判断句子语法可接受性	8,500	1,000
SST-2	单句分类	判断电影评论情感极性	67,000	800
MNLI	句子对分类	判断假设与前提的逻辑关系	393,000	10,000
MRPC	句子对分类	判断两个句子是否语义等价	3,700	400
QQP	句子对分类	判断两个问题是否语义重复	363,000	40,000
QNLI	句子对分类	判断句子是否包含问题答案	105,000	5,500
RTE	句子对分类	判断一个句子是否蕴含另一个句子	2,500	300
WNLI	句子对分类	解决代词消解问题	600	100
STS-B	语义相似度回归	评估两个句子间的语义相似度	5,700	1,500

4.2 评估标准

本实验中，模型在 GLUE 基准测试各项任务上的性能评估采用了多种标准化的指标，以全面衡量其在不同语言理解维度上的表现。以下详细介绍了这些主要评估指标的计算公式及其在实验中的应用：

1. 准确率 (Accuracy)

定义：准确率衡量了模型正确预测的样本数占总样本数的比例。它适用于多分类任务，表示模型分类的整体正确性。

应用场景：MNLI、SST-2、QNLI、RTE 等分类任务的主要评估指标。对于 MRPC 和 QQP 任务，准确率也作为其中一个评估指标被计算。

2. F1 分数 (F1 Score)

定义：F1 分数是精确率（Precision）和召回率（Recall）的调和平均值。它在处理类别不平衡的数据集时比单纯的准确率更能反映模型的性能，尤其适用于二分类或多分类的加权平均计算。

计算公式：

$$F1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

应用场景：MRPC 和 QQP 任务的关键评估指标之一。

3. 马修斯相关系数 (MCC)

定义：MCC 是一个衡量二分类器质量的指标，其值介于 -1 和 +1 之间。+1 表示完美预测，0 表示随机预测，-1 表示完全不一致的预测。

计算公式：

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

其中，TP (True Positives) 为真阳性，TN (True Negatives) 为真阴性，FP (False Positives) 为假阳性，FN (False Negatives) 为假阴性。

应用场景：CoLA 任务的专用评估指标。

4. 皮尔逊相关系数 (Pearson Correlation Coefficient)

定义：皮尔逊相关系数衡量了两个变量之间线性关系的强度和方向。其值介于 -1 和 +1 之间，+1 表示完全正线性相关，-1 表示完全负线性相关，0 表示没有线性相关。

$$\rho_{X,Y} = \frac{\text{cov}(X,Y)}{\sigma_X \sigma_Y} = \frac{E[(X - \mu_X)(Y - \mu_Y)]}{\sqrt{E[(X - \mu_X)^2]} \sqrt{E[(Y - \mu_Y)^2]}}$$

其中，X 和 Y 是两个变量， $\text{cov}(X,Y)$ 是它们的协方差， σ_X 和 σ_Y 分别是它们的标准差， μ_X 和 μ_Y 分别是它们的均值，E 表示期望值。

应用场景：STS-B 任务的评估指标之一，用于衡量模型预测的相似度分数与真实相似度分数之间的线性相关性。

5. 斯皮尔曼相关系数 (Spearman Correlation Coefficient)

定义：斯皮尔曼相关系数衡量了两个变量之间单调关系的强度和方向，不要求关系是线性的。它通过计算变量的秩次（排名）之间的皮尔逊相关系数来评估。其值也介于 -1 和 +1 之间。

计算公式：

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

其中， d_i^2 是第 i 个数据对的秩次差平方，n 是数据对的数量。

应用场景：作为 STS-B 任务的评估指标，用于衡量模型预测的相似度分数与真实相似度分数之间的单调一致性。

4.3 实验环境

表 1 硬件环境配置表

配置名称	配置类型	配置规格
CPU	Intel Xeon	2 核
内存	DDR4 ECC RAM	12GB
磁盘	SSD	80GB
GPU	NVIDIA Tesla (T4)	1 块

表 2 软件环境配置信息表

软件类型	软件版本
操作系统	Ubuntu 18
虚拟化服务	容器化环境
Spark 服务	PySpark
Python 版本	Python 3.x
深度学习框架	PyTorch / TensorFlow
GPU 加速库	CUDA / cuDNN

5 模型训练与评估

5.1 模型初始化与数据预处理

本实验阶段主要关注深度学习文本分类模型的构建与输入数据的准备。选用 `DistilBERT-base-uncased` 作为预训练模型，其优势在于通过知识蒸馏在保持性能的同时大幅提升了效率，非常适合大规模分布式环境。

针对 GLUE 基准测试中的各类下游任务（包括单句分类、句对关系识别及多分类推理），对 `DistilBERT` 预训练编码器进行微调。微调过程涉及在编码器之上附加任务专属的输出层（如线性分类头或回归头），并利用对应任务的标注训练数据，通过反向传播迭代更新模型参数，使模型能精准适配特定任务。

在输入数据处理方面，采用任务感知的分词逻辑。通过 `AutoTokenizer` 加载分词器，并根据任务类型（单句或句对）选择相应的文本字段进行编码。所有输入均经过截断和填充，统一至最大长度（128）。同时，针对字符串标签，将其转换为模型所需的整数格式，并对异常标签进行忽略处理。

最终，所有预处理完成的数据集被规范化为 `Hugging Face DatasetDict` 格式，并转换为 `PyTorch` 张量，明确了 `input_ids`、`attention_mask` 和 `labels` 等模型输入字段。这些步骤均在分布式计算环境中完成，确保了数据结构的一致性和训练质量。

5.2 模型训练流程

本实验的模型训练流程可概括为三个相互衔接的阶段：

首先是环境配置与模型初始化。代码部署于 `Google Colab` 环境，并采用挂载 `Google Drive` 的策略，有效解决了云端存储与运行时环境的限制，保障了模型、缓存及日志文件的持久化。为提升训练效率并降低对外部网络资源的依赖，项目禁用了 `wandb` 等报告工具。在模型选择上，以 `distilbert-base-uncased` 作为基础预训练模型，并根据 GLUE 基准测试任务的特点，对任务进行了分组配置。数据集

被分为了三大组：推理组（如 MNLI）、相似性组（如 QQP）和单句组（如 SST-2），以更好地管理和训练不同类型的任务。

其次为数据加载与预处理。此阶段的核心在于 `load_and_tokenize_for_training` 函数，该函数利用 `datasets` 库自动化地加载 GLUE 数据集，并实现了任务感知的分词逻辑。该逻辑能够智能地识别不同任务数据中的文本字段差异，通过 `AutoTokenizer` 进行分词，并统一应用截断与填充策略，将所有输入序列标准化至 128 的最大长度。此外，函数还负责标签的映射与格式化，将字符串标签转化为整数，并将处理后的数据集组织为 PyTorch 张量格式，为后续的模型训练奠定了坚实基础。

最后是模型微调与评估。`train_model_for_task` 函数首先加载预训练的 DistilBERT 模型，并根据任务的标签数量动态初始化分类头。`TrainingArguments` 类被用于精细化配置训练超参数，例如学习率、批次大小及训练轮次（针对特定任务进行了调整），同时启用了混合精度训练以加速计算。在评估环节，`compute_metrics` 函数针对不同任务类型定制了多样化的评估指标，涵盖了回归任务的皮尔逊/斯皮尔曼相关系数、CoLA 任务的马修斯相关系数，以及其他分类任务的准确率和 F1 分数。最终，`Trainer` 类封装了完整的训练循环，在每个周期结束后进行性能评估，并依据预设的最佳指标保存最优模型，从而确保了微调过程的有效性和稳定性。

5.3 结果分析

综合来看，模型在不同 GLUE 任务上的性能表现与数据集的规模和任务本身的复杂性密切相关。

处理大规模数据集时，模型展现出卓越的泛化能力。例如，在训练集高达 393K+ 的 MNLI 任务中，模型的验证准确率达到 82.28%，尽管验证损失略有波动，但准确率的稳步提升表明模型在海量数据中有效捕捉了逻辑推理模式。同样，在拥有 363K+ 训练样本的 QQP 任务上，模型取得了超过 90% 的准确率和 F1 分数，这得益于充足的数据量使其能够学习区分问题重复性的复杂模式。面对中等规模数据集，模型同样表现出色。在拥有 67K+ 训练样本的 SST-2 情感分析任务中，模型验证准确率超过 90%，证明了其在单句情感极性判断上的高效性。而在需要评估句子语义相似度的回归任务 STS-B 上，模型凭借 5.7K 训练样本取得了高达 0.87 左右的皮尔逊和斯皮尔曼相关系数，这充分展示了其量化深层语义理解的强大能力。

然而，对于小规模数据集，模型性能则受到了一定挑战。例如，在训练集仅有 2.5K 的 RTE 任务中，尽管训练轮次增加，但验证损失在后期显著上升，准确率提升缓慢，反映出模型在小样本上过拟合的风险。与之类似，MRPC 任务尽管数据量较小，但其相对简单的任务特性使模型依然取得了 83% 以上的准确率和

F1 分数。

值得注意的是，在最具挑战性的 CoLA 任务中，尽管有 8.5K 训练样本，模型的 MCC 分数仍相对较低，仅为 0.5357。这反映出该任务对语法可接受性的细微判断极为敏感，即使是强大的预训练模型，在面对此类高度抽象和主观的任务时，其性能也存在一定局限性。

表 1 GLUE 基准测试任务性能概览

名称	主要评估指标	最终性能结果	分析/结论
MNLI	Accuracy	0.823	泛化能力强，后期有轻微过拟合趋势
QQP	Accuracy, F1-score	Acc: 0.902, F1: 0.903	性能优异，得益于充足数据量
SST-2	Accuracy	0.901	表现出色，对情感极性判断高效
STS-B	Pearson, Spearman	Pearson: 0.873, Spearman: 0.870	语义量化能力强，相关系数高
CoLA	MCC	0.536	任务难度高，模型性能存在局限性
MRPC	Accuracy, F1-score	Acc: 0.838, F1: 0.833	任务相对简单，性能良好
RTE	Accuracy	0.643	数据量过小，易过拟合

图 3 以直观的直方图形式，全面展示了 DistilBERT 模型在 GLUE 基准测试任务上的性能概况。图中的每个任务对应一个或多个条形图，其高度代表了相应的评估指标值，而图例则清晰地标识了不同的评估指标类型，包括准确率、F1 分数、皮尔逊相关系数、斯皮尔曼相关系数以及马修斯相关系数。

通过分析此图，我们可以得出以下结论：首先，模型的性能在不同任务间存在显著差异。在 QQP、MRPC 和 SST-2 等任务中，模型的准确率和 F1 分数均超过 0.8，表明其表现优秀；但在 RTE 和 QNLI 等任务上，准确率则相对较低，仅在 0.6 左右，这反映出任务本身难度和数据特点对模型性能的影响。其次，模型在回归任务上表现出色，例如在 STS-B 任务中，皮尔逊和斯皮尔曼相关系数均超过 0.85，这证明了 DistilBERT 在量化句子语义相似度方面的强大能力。最后，对于 CoLA 这类具有挑战性的任务，模型的 MCC 分数显著偏低，仅为 0.5 左右，这进一步证实了该任务的复杂性以及即使是强大的预训练模型也存在性能局限性。

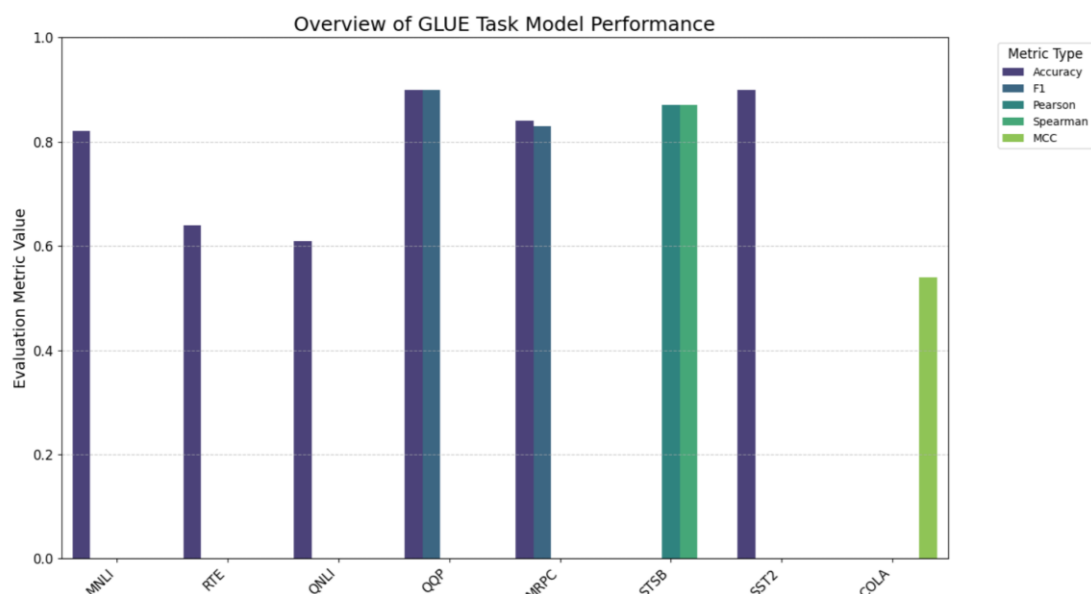


图3 DistilBERT 模型在 GLUE 基准测试任务上的性能概览

6 结语

本研究成功地对预训练的 DistilBERT 模型在 GLUE 基准测试的多个通用语言理解任务上进行了微调和评估。实验结果表明，DistilBERT 作为一款轻量级模型，在大多数 GLUE 任务中展现出令人满意的性能，尤其在拥有大规模训练数据支持的任务（如 MNLI、QQP、SST-2）上，其准确率和相关系数均达到了较高水平，充分验证了其强大的语言理解和迁移学习能力。然而，实验也揭示了模型在面对数据集规模较小（如 RTE）或任务本身难度较高（如 CoLA）的挑战时，性能提升空间有限，且易出现过拟合现象。这提示我们在实际应用中，需根据具体任务的数据特性和复杂程度，对模型选择和训练策略进行更精细的调整。

综上所述，DistilBERT 在通用语言理解任务中展现了显著的潜力和效率，为资源受限环境下的自然语言处理应用提供了可行方案。未来的研究工作可进一步探索更优化的超参数配置、引入更先进的正则化技术、或结合多任务学习范式，以期在保持模型轻量化的同时，有效提升其在各类复杂和数据稀疏任务上的泛化能力和鲁棒性。

参考文献

- [1] 冯贵兰,周文刚.基于 Spark 平台的并行 KNN 异常检测算法[J].计算机科学,2018,45(S2):349-352,366.

- [2] 孙建旺,吕学强,张雷瀚.基于词典与机器学习的中文微博情感分析研究[J].计算机应用与软件,2014,31(7):177-181.
- [3] 邢长征,李珊.文本情感分析的深度学习方法[J].计算机应用与软件,2018,35(8):102-106.
- [4] 杨立月,王移芝.Spark 在集成学习文本情感分析中的应用[J].计算机应用与软件,2020,37(6).